Evaluating Early Stopping and Pre-filtering Optimizations for Eclat and FP-Growth

Landon Hadre[1], Ethan Konkolowicz[1], Abdulrauf Gidado[2], and Faria Khandaker[2]

[1] University of Windsor, 401 Sunset Avenue, Windsor, ON N9B 3P4, Canada
{hadrel,konkoloe}@uwindsor.ca
[2] Algoma University, 1520 Queen St E, Sault Ste. Marie, ON P6A 2G3, Canada
{abdulrauf.gidado,faria.khandaker}@algomau.ca

Abstract. Frequent pattern mining is essential for discovering meaningful associations in large datasets, with applications in market basket analysis, recommendation systems, and decision support. While Eclat and FP-Growth are widely used algorithms for this task, they face performance limitations with large datasets or low support thresholds. This paper evaluates two optimization strategies: an early stopping technique for Eclat that skips itemsets below the minimum support threshold during recursion, and a pre-filtering approach for FP-Growth that removes infrequent items before tree construction. We tested both optimizations on synthetic datasets with realistic properties including Zipf-distributed item popularity, correlated purchase themes, and varying transactions densities. The Eclat optimization achieved an average of 33% improvement across all configurations, with gains up to 72% on sparse datasets at high support threshold. Whereas FP-Growth pre-filtering saw degraded performance by an average of 18% as scanning transactions outweighed the benefits of constructing a smaller tree. These findings demonstrate that effective optimizations must reduce more computation than they introduce and that intuitive improvements won't always yield gains.

Keywords: Frequent Pattern Mining · Eclat · FP-Growth.

1 Introduction

Frequent pattern mining is a fundamental task in data mining and knowledge discovery, aimed at identifying patterns, correlations, and associations in large datasets. These patterns represent meaningful and actionable knowledge, help support applications such as market basket analysis, recommendation systems, web usage mining, and decision support. With big data and data warehousing, frequent mining helps reveal hidden structures in high-dimensional data and allows for more efficient analytical processing.

Classic algorithms such as Eclat and FP-Growth are widely used due to their effectiveness in mining frequent itemsets without candidate generation. However, they face performance and scalability challenges when applied to large datasets

or when the minimum support threshold is low. Eclat's depth first search can lead to excessive memory consumption, while FP-Growth may incur significant overhead when constructing trees for many infrequent items. Addressing these limitations is essential to make frequent pattern mining practical and efficient in real world environments.

This paper proposes enhancements to both Eclat and FP-Growth that improve memory efficiency, reduce computational overhead, and enable scalable frequent pattern mining for large and low-support datasets.

2 Related Work

2.1 FP-Growth Algorithm

The paper titled "A Systematic Review of Frequent Itemset Mining Algorithms and Their Capabilities" by Anshu Singla and Parul Gandhi provides a comprehensive review of various Frequent Itemset Mining algorithms including FP-Growth, MMFI, Max-IFP, COFI, and Parallel FP-Growth [1].

The original FP-Growth algorithm was proposed by Han et al. [3]. It compresses the entire database into an FP-Tree with just one scan, reducing the need for multiple database scans like the Apriori algorithm. The tree structure allows for efficient mining of frequent patterns without generating candidate itemsets. MMFI (Modified Matrix Frequent Itemsets) uses FP-Arrays and FP-Matrices to enhance memory usage and mining speed. Max-IFP (Maximal Frequent Patterns) employs an IFP-Tree to reduce the space required for storing large itemsets. The COFI algorithm uses a pruning strategy to reduce memory usage by building a COFI-Tree. This algorithm is most effective for high-speed transactional databases but struggles with sparse datasets [1].

Parallelization efforts in the FP-Growth algorithm were also covered in the literature. Algorithms like Parallel FP-Growth (PFP) are used to distribute the mining process across multiple nodes to improve performance. Newer algorithms integrate MapReduce frameworks to handle large datasets more effectively [1].

The limitations of these approaches are notable. The original FP-Growth algorithm requires significant memory to construct the FP-Tree, especially when the data has many unique items. This can lead to complex branches in the tree and result in large memory usage. The COFI algorithm struggles with sparse datasets where itemsets are less likely to overlap, causing the FP-Tree structure to become inefficient if the data is too diverse. Parallel algorithms like PFP and MapReduce-based methods have issues with load balancing and communication overhead, leading to reduced scalability. These flaws affect all algorithms relying heavily on FP-Trees, causing difficulty scaling when the tree becomes too deep or complex [1].

Our implementation of the FP-Growth algorithm applies a pre-filtering optimization inspired by techniques like COFI-Tree pruning. Similar to the pruning methods discussed in the literature, our code removes infrequent items before building the tree by checking the frequency of items before constructing the

FP-Tree. This approach theoretically reduces memory usage and computational overhead by not constructing branches for infrequent items.

2.2 Eclat Algorithm

The paper titled "Boosting Frequent Itemset Mining via Early Stopping Intersections" by Huu Hiep Nguyen proposes a technique to improve the efficiency of existing depth-first search-based itemset mining algorithms, such as Eclat/dEclat and PrePost+ [2].

The Eclat algorithm uses a vertical database format, storing each item with the list of transaction IDs (TID-lists) in which it appears. Candidate itemsets are generated by intersecting these transaction ID lists, and infrequent items are filtered out based on the minimum support threshold. This technique is designed to be universally applicable to various depth-first search algorithms that use TID-list, Diffsets, or N-list methods [2].

One of the key contributions of Nguyen's paper was an improvement to the Eclat algorithm using an Early Stopping Technique. The paper introduces an Early Stopping criterion for checking support of candidate itemsets during list intersections. It identifies infrequent itemsets early on, cutting down unnecessary computations and comparisons. The method was tested on several datasets, showing noticeable improvements in runtime, especially for datasets with a high ratio between the number of candidates and frequent itemsets [2].

The improved algorithm has several limitations. First, the Early Stopping technique adds a small overhead when dealing with frequent itemsets since the stopping checks themselves take time to execute. Second, some datasets like Accidents, Chess, Connect, and Pumsb did not benefit much from the Early Stopping technique due to their data structure (low ratio of candidates to frequent itemsets). Third, the improvements are not applicable to all algorithms since the technique is designed specifically for algorithms that rely on list intersections and is not applicable to algorithms like Apriori or FP-Growth [2].

Our implementation of the Eclat algorithm incorporates the Early Stopping technique proposed by Nguyen. The optimization checks support thresholds earlier during recursive intersections and avoids unnecessary calculations for itemsets that are known to be infrequent. Our implementation provides a toggleable improved mode which activates the Early Stopping feature, making the algorithm adaptable for both standard Eclat usage and more efficient mining when needed.

3 Proposed Optimizations

This Section describes the two optimizations we implemented and tested, an early stopping technique for the Eclat algorithm and a pre-filtering approach for the FP-Growth algorithm. Both optimizations aim to reduce unnecessary computation by identifying and skipping infrequent itemsets earlier in the mining process.

3.1 FP-Growth

The standard FP-growth algorithm builds the FP-Tree by scanning transactions and filtering infrequent items during the tree construction. Our optimizations attempt to improve this by adding a pre-filtering step before tree construction. Our approach works as follows:

- First pass: Scan all transactions and count item frequencies.
- Filter step: For each transaction, remove any items that appear less than the minimum support threshold.
- Second pass: Build the FP-Tree using only the filtered transactions.

The idea is that by removing infrequent items from transactions before building the tree, we would create a smaller tree structure with fewer branches. This should theoretically reduce memory usage and speed up construction of the tree.

3.2 Eclat

The standard Eclat algorithm performs depth-first search using TID-list intersections. For each candidate itemset, it intersects the transaction ID list of its component items to determine support (the size of its TID-list) is already below the minimum support threshold. If so, the algorithm skips that branch entirely and moves to the next candidate. Specifically, the improvement works as follows:

- During recursion, before processing an itemset, check if its TID-list size is less than minimum support
- if yes, skip this itemset completely (continue to next)
- if no, proceed with normal intersection and recursive calls

This avoids wasting computation on itemsets that are guaranteed to be infrequent. The overhead is minimal (one comparison per itemset), but the savings can be significant when many candidates fall below the threshold.

4 Experimental Setup

4.1 Datasets

We evaluate Eclat and FP-Growth on synthetic transactional datasets generated to mimic market-basket style data with varying density. Each transaction is a set of items labelled I0001 - IXXXX

We generate three dataset modes:

- **Sparse:** Short transactions (few items/transaction)
- **Mixed:** Moderate transaction lengths with balanced correlation and moderate noise. This should represent average retail patterns
- **Dense:** Longer transactions with higher correlation and more noise items. This should simulate bulk shopping scenarios like warehouse or company spending.

4.2 Realistic Data Characteristics

To ensure our synthetic data reflects real world patterns, we implemented multiple statistical properties to our set:

Zipf Distribution: Item popularity follows Zipf distribution (a = 1.15), where a small number of items appear frequently whereas the rest are rare. This matches a real dataset as bestsellers dominate sales.

Correlated Themes: Items are organized into 8-35 themes (depending on the size of the dataset) of 4-10 items each. Transactions have a 65-85% chance of including items from the same theme, simulating real purchasing patterns such as dinner or birthday supplies.

Pareto Transaction Lengths: Transaction lengths follow a Pareto distribution, creating realistic variation where most transactions are short but some are significantly longer.

Negative Correlations: Certain item pairs are marked as mutually exclusive (10-80 pairs depending on dataset size), simulating real patterns where customers choose one brand or the other but rarely both.

Rare Items: 3% of transactions include items from a "rare pool" (bottom 8% of items by popularity), to allow the algorithms to handle infrequent itemsets.

4.3 Dataset sizes

We tested four dataset scales across three density modes (sparse, mixed, dense), producing 12 total test configurations.

Table 1. Dataset Sizes

Transactions	Items	Description
100	50	Small-scale validation
1,000	100	Small dataset
10,000	200	Medium dataset
100,000	500	Large dataset

For each dataset we tested two minimum support thresholds: 2% of transactions which is a low threshold that can discover more frequent sets. Then 10% of transactions which is higher and can find fewer but more significant patterns. Algorithms were implemented in C++17 and compiled using g++ with -O3 optimization. Each test configuration was executed 3 times and averaged to reduce variance. Tests were conducted on a laptop with M4 chip, 16GB RAM, running Sequoia 15.6.1.

5 Results

5.1 Eclat Performance

The early stopping optimization we implemented showed consistent improvements across all dataset modes and sizes. The average was a 43.1% improvement

for sparse datasets, 36.7% for mixed, and 19.6% for dense sets. The best performance gains occur when the dataset is sparse with higher support thresholds. The highest improvement was 72.03% with 100,000 transactions with 10% support on the mixed dataset. Followed by 100,000 transactions and 10% support with an improvement of 71.43%.

A recurring theme was higher support thresholds consistently produced better improvements than the lower support. This is due to a higher threshold means more items fall below the minimum support which gives the early stopping mechanism more opportunities to skip unneeded computations. Dense datasets showed the smallest improvements, with two cases showing slight slowdowns: mixed 100 transactions at 2% support (-3.23%) and dense 100 transactions at 2% support (-1.47%). This occurs because dense data has more frequent itemsets, leaving fewer opportunities for early stopping. Figure 1 illustrates these improvements across sparse datasets.

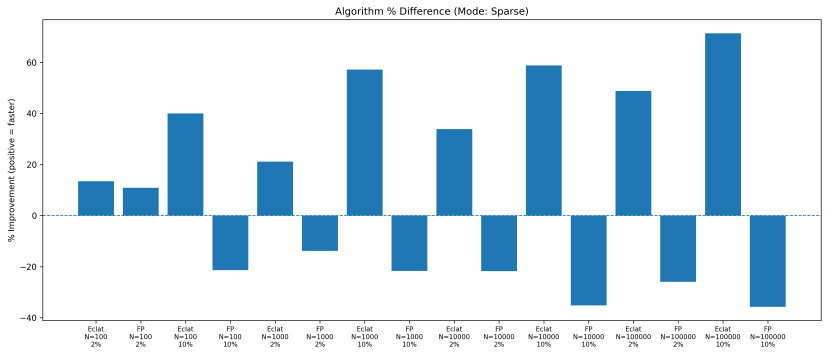

Fig. 1. Performance improvement on sparse datasets. Positive values indicate faster execution with the optimized algorithm.

5.2 FP-Growth Performance

The pre-filtering optimization we implemented showed consistent degradation for performance across practically all test cases. The average slowdown was 21.9% for sparse datasets, 20.9% for mixed datasets, and 10.5% for dense datasets. The worst performance was observed on mixed datasets at 10% support, with slowdowns reaching 37.07% on the 100,000 transactions dataset. Sparse datasets at 10% support also saw poor performance with the largest dataset showing a 35.75% slowdown. Only one test case saw improved performance: dense 100,000 transactions at 2% support achieved a 2.18% performance boost. This might suggest pre-filtering may only benefit large, dense datasets where removing infrequent items significantly reduces the tree complexity. The consistent slowdown confirms that scanning the entire data set to pre-filter items before construction

of the tree adds more overhead than it saves during tree building. As shown in Figure 2, this pattern holds across mixed density datasets.

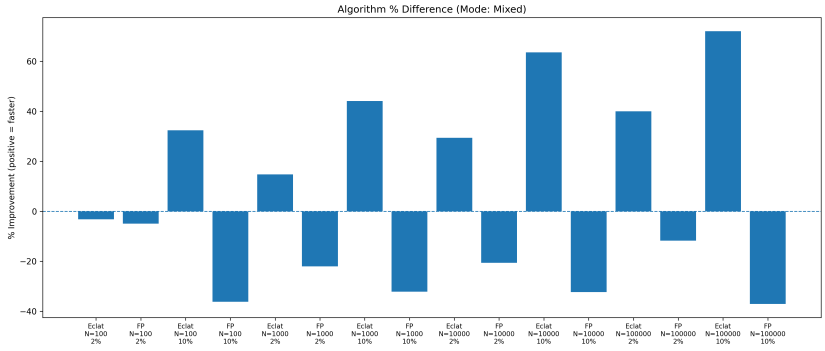

Fig. 2. Performance comparison on mixed datasets showing Eclat improvements versus FP-Growth degradation.

6 Discussion

The contrasting results between Eclat and FP-Growth shows that optimizing an algorithm in one area might add computational overhead in another area. The Eclat early stopping optimization succeeded because it adds minimal overhead (one comparison per item set) while potentially saving significant work by skipping entire recursive branches. The improvement scales with the number of infrequent itemsets, explaining why sparse data and higher support thresholds showed the best results. When most itemsets are infrequent there are more branches to prune. The attempted FP-Growth pre-filtering optimization failed due to it needing a full pass through of all transactions before tree construction. The standard FP-Growth algorithm already filters infrequent items during tree construction, so our pre-filtering essentially duplicates this work. The extra memory allocation for filtering these transactions adds too much unneeded overhead. Our results align with Nguyen's findings on early stopping for Eclat, which had reported improvements on datasets with high ratios of candidates to frequent itemsets. Our sparse datasets match this profile, showing improvements up to 71%. However, Nguyen noted that datasets with low candidate ratios showed minimal benefits, similar to our dense dataset. Figure 3 shows this pattern continues with dense datasets.

The FP-Growth result suggest that optimizations targeting tree construction should focus on the construction process itself rather than pre-processing. COFI-tree pruning, which operates during mining rather than before construction, may be more effective.

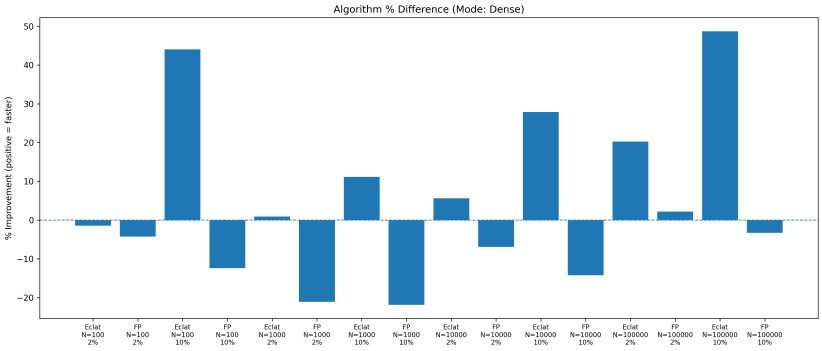

Fig. 3. Performance comparison on dense datasets confirming Eclat gains and FP-Growth overhead across all density modes.

7 Conclusion

This paper evaluated two optimization strategies for frequent pattern mining algorithms: early stopping for Eclat and pre-filtering for FP-Growth. The Eclat early stopping optimization proved effective, achieving an average improvement of 33% across all test configurations, with gains up to 72% on sparse datasets with high support thresholds. The optimization works by checking support thresholds before processing each item set. avoiding unnecessary intersection operations on itemsets guaranteed to be infrequent. The FP-Growth pre-filtering optimization did not improve performance as it showed an average slowdown of 18% across all tests. The overhead of scanning transactions to remove infrequent items before tree construction outweighed any benefits from building a smaller tree. These results demonstrate that not all intuitive optimizations improve performance. The key factor is whether the optimization reduces more work than it introduces. Early stopping succeeds because it adds minimal overhead while potentially pruning large portions of the search space. Pre-filtering fails because it duplicates work already performed during standard tree construction. Future work could explore alternative FP-Growth optimizations that operate during tree construction rather for pre-processing. Additionally, testing on real-world datasets from domains like retail or web streams would validate whether these findings generalize beyond synthetic data.

References

1. Singla, A., Gandhi, P.: A Systematic Review of Frequent Itemset Mining Algorithms and Their Capabilities. EasyChair Preprint no. bS8f (2024). https://easychair.org/publications/preprint/bS8f
2. Nguyen, H.H.: Boosting Frequent Itemset Mining via Early Stopping Intersections. arXiv preprint:1901.07773 (2019). https://arxiv.org/abs/1901.07773

3. Han, J., Pei, J., Yin, Y.: Mining Frequent Patterns without Candidate Generation. In: Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data, pp. 1–12. ACM, New York (2000)